

FluxPlan – Adaptive Engine & UX/Naming Notes (zum Nachlesen)

Für eine **UI-zentrierte** Übersicht (was Nutzer sehen, Trigger, Baseline vs. Adaptive): [UI-FEATURES-KATALOG.md](#).

Dieses Dokument sammelt **konkret aus dem Code**:

- **Welche adaptiven Features es gibt** (Regeln/Heuristiken)
 - **Was sie tun** (Suggestion-Typen, Datenänderungen, Undo)
 - **Wie/wo die Engine getriggert wird** (UI → API → Engine)
 - **Wie du sie in der Demo sofort triggern kannst**
 - **UX-Verbesserungen** (Fokusliste Done-Handling, Done-Tasks wiederfinden)
 - **Naming-/Redundanz-Funde** (Menü, Suggestion-Texte, Begriffe)
-

1) Adaptive Features: Wieviele, wo, was genau?

1.1 Anzahl: 7 Regeln in der Engine

Die Engine führt **7 Regeln** aus, fest verdrahtet in:

- `src/lib/adaptive/adaptiveEngine.ts` (Array rules)

Aktive Keys (Reihenfolge = Ausführungsreihenfolge):

- `view_preference`
- `reminder_preference`
- `daily_focus`
- `calendar_conflict`
- `adaptive_task_creation`
- `adaptive_optional_fold`
- `adaptive_optional_unfold`

Ob eine Regel aktiv ist, hängt zusätzlich von DB-Flags in `AdaptiveRule` ab (per API togglebar) und vom Master-/Level-Config. Neue Regeln müssen in `ensureAdaptiveRules / Seed / Demo-ensureRules` stehen, sonst fehlt der FK in `AdaptiveSuggestion`.

1.2 Master-Schalter + Intervention-Level + Pausenlogik

Quelle:

- `src/lib/adaptive/engineConfig.ts`
- `src/app/api/preferences/route.ts`
- `src/components/settings/preferences-form.tsx`

Mechaniken:

- **Master-Schalter:** Preference `adaptive.enabled`
 - Wenn `false`: Engine erzeugt **keine** Suggestions (`runAdaptiveEngine` logged `adaptive_disabled`).
- **Intervention-Level:** Preference `adaptive.interventionLevel` (0–3)
 - 0: Engine effektiv aus (über `isRulePaused`).
- **Snooze Pause (generisch):** wenn eine Suggestion snoozed ist, gilt die Rule für ein konfigurierbares Fenster als pausiert (Standard **24 h**, in DEMO_MODE typisch **10 min** — `FP_SNOOZE_MS` in `engineConfig.ts`). **reminder_preference ist ausgenommen** (eigene Pause über Preferences, siehe unten).
- **Erinnerungs-Vorschlag vertagt:** POST `.../respond` mit `snooze` bei `reminder_preference` setzt `adaptive.reminderSuggestionSnoozeUntil` (lokaler Kalendertag + **N** Tage; **N** aus `adaptive.reminderSnoozeDays`, Standard **3**, UI: Personalisierung). Die Regel `reminder_preference` wertet `until` aus und erzeugt bis dahin **keine** neuen Vorschläge.
- **Reject Cooldown:** wenn innerhalb des Cooldown-Fensters ≥ 2 Rejects für dieselbe Rule auftreten (Standard **14 Tage**, in Demo verkürzbar über `FP_COOLDOWN_*`):
 - setzt Preference `rule.cooldown.<ruleKey>` bis +14 Tage
 - Implementierung: `maybeApplyCooldownAfterReject()` in `engine-Config.ts`
- **Spam-Prevention (Pending Dupe):** pro (`userId`, `ruleKey`, `type`) wird kein zweiter pending Suggestion angelegt:
 - `adaptiveEngine.ts` prüft vor `create` auf vorhandenes pending.

1.3 API-Lifecycle: Suggestions (GET + respond + Undo)

Quellen:

- GET `/api/suggestions` → `src/app/api/suggestions/route.ts`
- POST `/api/suggestions/[id]/respond` → `src/app/api/suggestions/[id]/resp`

Actions:

- `accept`, `reject`, `snooze`, `undo`
- Statuswechsel: `pending` → `accepted/rejected/snoozed/undone`
- Logging: `taskInteraction` Einträge wie `suggestion_accepted`, `suggestion_rejected`, `suggestion_snoozed`, `suggestion_undone`

Wichtig (Undo-Reality-Check):

- “Echte” Daten-Reversion / Preference-Reset ist implementiert für:
 - `type === "start_view"` → löscht `startView`

- `type === "reminder_suggestion"` → setzt `Task.reminderAt = null` (löscht auch `adaptive.reminderSuggestionSnoozeUntil`)
 - `type === "task_form_chips"` → löscht `UserPreference adaptive.taskFormChips`
 - `type === "task_form_optional_fold"` → löscht `UserPreference taskFormOptionalFold`
 - `type === "task_form_optional_unfold"` → setzt `taskFormOptionalFold` wieder `{ enabled: true }` (Rückkehr zum eingeklappten Zustand nach Undo eines Ausklapp-Vorschlags)
 - Für alle anderen Types: undo markiert primär Status undone + Logging, ohne dass Daten zwingend zurückgesetzt werden.
-

2) Was macht jede Regel/Heuristik? (praktische Wirkung)

2.1 view_preference → Startansicht vorschlagen

Quellen:

- Regel: `src/lib/adaptive/rules/viewPreferenceRule.ts`
- Apply/Undo: `src/app/api/suggestions/[id]/respond/route.ts` (`applySuggestion/undoSuggestion`)
- Start-Redirect: `src/lib/nav/resolve-start-redirect.ts` + `src/lib/settings/startView.ts`

Praktische Wirkung:

- Engine erzeugt i. d. R. einen Suggestion mit `type: "start_view"` (so gebaut, dass `respond` ihn anwenden kann).
- **Accept** setzt Preference `startView` (`Upsert, value { href }`).
- **Undo** löscht `startView`.
- UI kann nach **Accept** sofort auf die neue Start-View navigieren (z.B. Today-Banner).

Begriffe im UI:

- In Code/Prefs heißt es `startView`. In der UI verwenden wir durchgehend „Startansicht“.

2.2 reminder_preference → Erinnerung vorschlagen

Quellen:

- Regel: `src/lib/adaptive/rules/reminderPreferenceRule.ts`
- Apply/Undo: `src/app/api/suggestions/[id]/respond/route.ts`

Praktische Wirkung:

- Suggestion type: "reminder_suggestion" mit `payload.taskId + payload.proposedReminderAt`.
- **Accept** setzt `Task.reminderAt`.
- **Undo** setzt `Task.reminderAt = null`.

2.3 daily_focus → Fokus-Hinweis (informational)

Quellen:

- Regel: `src/lib/adaptive/rules/dailyFocusRule.ts`
- Banner/UX: `src/components/adaptive/pending-suggestion-banner.tsx` (global)

Praktische Wirkung:

- Engine kann Suggestion `ruleKey: "daily_focus", type: "daily_focus"` erzeugen.
- In TodayDashboard ist `daily_focus` explizit als **informational-only** behandelt:
 - der "Accept" Button ist "Verstanden" und soll **keine Task-Daten ändern**.
- Die tatsächliche **Fokusliste** wird clientseitig aus offenen Tasks abgeleitet (`deriveTodayBuckets()`), nicht aus Engine-Payload.

2.4 calendar_conflict → Konflikthinweis

Quellen:

- Regel: `src/lib/adaptive/rules/calendarConflictRule.ts`
- UI Konflikterkennung: `src/components/planning/week-planner.tsx` (`detectConflicts, ConflictDetailsCard`)

Praktische Wirkung:

- UI markiert Konflikte immer (Overlaps) – unabhängig davon, ob Engine etwas vorschlägt.
- Engine-Regel kann zusätzlich Suggestions erzeugen (Details siehe Regeldatei).

2.5 adaptive_task_creation → Vorschlags-Chips (Muster in optionalen Feldern)

Quellen:

- Regel: `src/lib/adaptive/rules/adaptiveTaskCreationRule.ts`

- Formular: `src/components/tasks/progressive-task-form.tsx`, `task-form-dialog.tsx`
- Preference lesen: `src/lib/settings/task-form-chips.ts` (`adaptive.taskFormChips`)
- Parser: `src/lib/parser/task-parser.ts`

Praktische Wirkung:

- Das Formular ist **ohne Engine** schon progressiv (Nutzer aktiviert Zusatzfelder per Chips; Parser füllt bei Bedarf).
- Die Regel feuert nach **screen: "task_created"**, wenn in den letzten Aufgaben **optionale Felder** (Kategorie, Tags, Dauer, Erinnerung, Beschreibung) über Schwellen hinweg **häufig** genutzt werden (Details + Sample-Größe in der Regeldatei; `thresholdMultipller` / Eingriffsstufe). **Gast-Pseudonyme G01 / G02**: Muster aus der zuletzt „reichen“ Aufgabe.
- Suggestion-type: **task_form_chips**, Payload u. a. `chipKeys`. **Accept** in `applySuggestion`: `upsert adaptive.taskFormChips { enabled: true, chipKeys }` — Formular blendet diese Chips beim Laden ein.
- **Baseline**: diese Suggestion entsteht praktisch nicht (Engine aus / keine Evaluations).

2.6 `adaptive_optional_fold` → Zusatzfelder standardmäßig einklappen (seltene Nutzung)

Quellen:

- Regel: `src/lib/adaptive/rules/adaptiveOptionalFoldRule.ts`
- Präferenz lesen: `src/lib/settings/task-form-optional-fold.ts` (`readTaskFormOptionalFold`)
- UI: `progressive-task-form.tsx`, `task-form-dialog.tsx`; Schalter: `preferences-form.tsx`
- Apply/Undo: `src/app/api/suggestions/[id]/respond/route.ts`

Praktische Wirkung:

- Feuert nach **screen: "task_created"**, wenn in den letzten Aufgaben **Kategorie, Tags, Dauer, Erinnerung oder Beschreibung** nur selten vorkommen (Anteil „reiche“ Optionalfelder unter einer Schwelle; Sample-Größe skaliert mit Eingriffsstufe).
- Kein Vorschlag, wenn **taskFormOptionalFold** schon aktiv, ein **task_form_chips**-Vorschlag noch **pending** ist, oder bereits ein **task_form_optional_fold** pending ist.
- Suggestion-type: **task_form_optional_fold**.
- **Accept**: UserPreference **taskFormOptionalFold** = { `enabled: true` } → Formular zeigt zunächst nur Kernfelder; „**Weitere Felder einblenden**“ klappt auf.

- **Undo:** Preference-Eintrag wird gelöscht.
- **Baseline:** Regel kann in Daten **nicht** ausgelöst werden, wenn keine `task_created`-Evaluations laufen; die **Einstellung** (Schalter) wirkt in Baseline trotzdem — reine UX-Präferenz ohne Vorschlag.

2.7 `adaptive_optional_unfold` → Zusatzfelder wieder ausklappen

Quellen:

- Regel: `src/lib/adaptive/rules/adaptiveOptionalUnfoldRule.ts`
- Apply/Undo: `src/app/api/suggestions/[id]/respond/route.ts`

Praktische Wirkung:

- Nur sinnvoll, wenn **`taskFormOptionalFold` bereits aktiv** ist und die Nutzung optionaler Felder wieder **steigt** (Anteil „reiche“ Aufgaben über Schwelle; Gast: letzte Aufgabe nutzt wieder Optionalfelder).
- Kein paralleler **`task_form_chips`**-Vorschlag pending.
- Suggestion-type: **`task_form_optional_unfold`**. **Accept:** löscht die Preference `taskFormOptionalFold`. **Undo:** setzt Einklappen wieder (`enabled: true`).

2.8 UI: Vorschläge unterscheidbar machen

Quellen:

- `src/components/adaptive/suggestion-visuals.ts` — `getSuggestionVisualMeta(ruleKey)` (Icon, Akzentfarbe, Badge-Text, Strapline)
- `src/components/adaptive/adaptations-tab.tsx` — Liste + Detailpanel mit linkem Rand, Badge, Strapline-Band
- `src/components/adaptive/pending-suggestion-banner.tsx` — globales Banner nutzt dieselbe Metadaten-Schicht

Zweck: Gleiche Button-Leiste („Annehmen / Nicht jetzt / Ablehnen“) darf **nicht** suggerieren, dass alle Vorschläge dieselbe *Art* von Wirkung haben — daher visuelle Kodierung pro `ruleKey`.

2.9 Zeitliche Überlappung beim Erstellen (nicht Engine)

Quellen:

- `src/lib/planning/task-time-overlap.ts` — `inferOverlapHintDurationMinutes`, `overlappingOpenTasksForDraft`
- `src/components/tasks/task-schedule-overlap-hint.tsx` — eingebunden in `progressive-task-form.tsx` und `task-form-dialog.tsx`

Hinweis: **reine Client-Warnung** bei Datum+Uhrzeit; kein Blockieren des Speicherns.
Fehlende Dauern: Herleitung aus offenen Aufgaben (siehe Katalog §3.5); **Kalender-Raster** (week-planner.tsx) nutzt weiterhin **45 min**-Fallback.

3) Wie wird die Engine getriggert? (Trigger-Karte)

3.1 Direkter Trigger: POST /api/adaptive/evaluate

Quelle:

- src/app/api/adaptive/evaluate/route.ts

Body:

```
{ "screen": "/heute", "taskId": null, "metadata": { "trigger": "manual" } }
```

3.2 Globales Banner + Evaluate (Hauptseiten)

Quelle:

- src/components/shell/app-shell.tsx + src/components/adaptive/pending-suggestion-banner.tsx

Bei **Adaptive** (nicht Baseline), sobald der Nutzer eine Hauptseite besucht (**nicht** /anpassungen, nicht / ohne Shell):

- POST /api/adaptive/evaluate mit { screen: <pathname> }
- GET /api/suggestions?status=pending und Auswahl einer „preferred“ Suggestion (wie zuvor auf /heute: u. a. daily_focus, view_preference)

3.3 Navigation/View-Change Trigger (opportunistisch)

Quelle:

- src/components/shell/app-shell.tsx → useLogViewChange(pathname)
- src/app/api/interactions/route.ts

Beim Route-Wechsel:

- UI sendet POST /api/interactions mit type:"view_changed", screen: pathname
- API ruft dann: runAdaptiveEngine({ userId, screen: parsed.data.screen, metadata })

Zusätzlich sendet UI parallel auch POST /api/events (nur Log; keine Engine).

3.4 Task Created Trigger

Quelle:

- `src/app/api/tasks/route.ts`

Nach dem Create:

- `runAdaptiveEngine({ userId, screen: "task_created", taskId })`
-

4) Demo: “sofort triggern” – zuverlässige Wege

4.1 1-Klick: Demo-Seed führt Engine-Evals direkt aus

Quelle:

- UI: `src/components/settings/demo-seed-button.tsx`
- API: `src/app/api/data/demo/route.ts`
- Rollen: `src/lib/demo/roles/*.ts`

Ablauf:

- `/einstellungen` → Demo-Setup → Rolle wählen → “Demo-Daten laden”

Was passiert im Backend:

- Seedet Tasks/Prefs/Interactions (optional Reset)
- Führt definierte `def.evaluations` aus (`runAdaptiveEngine` pro Evaluation)
- Führt danach **immer** noch `runAdaptiveEngine({ screen: "/heute" })` aus (“populate focus/view suggestions”)

Empfehlung:

- Rolle `evalrunner` ist am ehesten dafür gebaut, Suggestion-Lifecycle + Konflikte schnell sichtbar zu machen.

`adaptive_optional_fold` auslösen (ohne Zufall):

- Viele Aufgaben **ohne** `listName`, **ohne** Tags, **ohne** `estimatedMinutes`, **ohne** `reminderAt`, **ohne** `description` anlegen (per API oder UI), bis die Mindestanzahl in der Regel erreicht ist; kein offener `task_form_chips`-Vorschlag.
- Danach eine weitere Aufgabe erstellen → `task_created` Evaluation.

4.2 “Ohne Demo” schnell sichtbar: ping-pong + Seitenwechsel

- Starte Session (Adaptive)

- Öffne z. B. /heute oder /aufgaben (Banner-Outlet triggert evaluate + lädt pending)
- Wechsle 4–5x zwischen /heute, /kalender und /aufgaben (triggert view_changed → Engine; Banner aktualisiert sich)

4.3 Kurz: Baseline vs. Adaptive (Engine-Sicht)

	Baseline
runAdaptiveEngine	wird oft
Neue AdaptiveSuggestion	praktisch
task_form_chips / task_form_optional_fold / task_form_optional_unfold	nein
Gast G01/G02 (isGuestStudyUser)	—
taskFormOptionalFold (manuell unter Einstellungen)	ja , reine

5) UX-Verbesserungen (aus deiner Anforderung)

5.1 Fokusliste: “Done klicken” soll bestätigen + nicht sofort “weg”

Ist-Stand:

- In TodayDashboard verschwindet ein Task nach Check sofort aus der Fokusliste, weil:
 - Checkbox triggert PATCH /api/tasks/[id] {status:"done"} und danach load(); load() lädt nur status=open (/api/tasks?status=open).
 - Quelle: src/components/planning/today-dashboard.tsx → FocusListItem.toggleDone()

Gewünschtes Verhalten (2 Optionen):

Option A (Minimal, sehr klar): Confirm-Dialog

- Vor dem PATCH: window.confirm("Wirklich erledigt?")
- Wenn “Abbrechen”: kein Statuswechsel.
- Vorteil: simpel, verhindert “versehentlich weg”.

Option B (besser UX): “Haken setzt, Task bleibt kurz sichtbar”

- Lokal im FocusListItem “pendingDone UI” (z.B. 1–3 Sekunden) oder “Undo”-Toast
- Task bleibt in der Liste mit Häkchen + Strikethrough, wird erst nach kurzer Zeit / Undo-Fenster entfernt.

- Dazu muss TodayDashboard entweder:
 - kurzzeitig lokale Overrides rendern, oder
 - zusätzlich done Tasks (limitiert) mitladen, damit sie nicht sofort aus status=open verschwinden.

Konkrete Code-Stelle:

- `src/components/planning/today-dashboard.tsx` → `FocusListItem.toggleDone()`

5.2 Done-Tasks: “ich finde sie sonst nie wieder”

Ist-Stand:

- TasksScreen kann Done/Archiv anzeigen (Status Filter existiert).
 - `Status = "all" | "open" | "done" | "archived"` in `src/components/tasks/tasks-screen.tsx`
- Default ist aber open (`useState<Status>("open")`) → Done sieht man “nicht von alleine”.

Empfehlungen:

- **Aufgaben-Seite** als “Wiederfinden-Ort” ist am sinnvollsten:
 - Default-Status auf `all` ändern, oder
 - prominente Quick-Filter/Buttons “Offen | Erledigt | Archiv” hinzufügen (statt im “Advanced” zu verstecken).
- Alternativ auf `/heute` eine kleine Sektion “Heute erledigt” (letzte 3) zeigen:
 - Dafür müsste `/api/tasks?status=done&...` geladen werden (oder `GET /api/tasks` ohne status + client filter).

Konkrete Code-Stellen:

- Default Status: `src/components/tasks/tasks-screen.tsx` (`const [status, setStatus] = useState<Status>("open")`)
- Server-Filter wird bereits unterstützt: `src/app/api/tasks/route.ts` (`Query param status`)

6) Naming / Redundanz / Verwirrungen (konkret gefunden)

Ziel: Begriffe konsistent machen, damit Nutzer nicht denken “das ist was anderes”, obwohl es dasselbe meint.

6.1 “Startansicht” vs “Standardansicht” (bereinigt)

Fundstellen:

- Willkommen-Text: `src/app/(app)/willkommen/page.tsx` (nutzt „Startansicht“)
- Suggestion/Rule/UI: `viewPreferenceRule` + weitere Texte nutzen „Startansicht“
- Preference-Key heißt `startView` (`src/lib/settings/start-view.ts`)

Empfehlung:

- Einen Begriff wählen (z.B. überall „Startansicht“) und UI-Text + Labels angleichen.

6.2 Mobile Menü: “Info” vs Desktop “Willkommen”

Fundstellen:

- Desktop Nav: `PRIMARY_NAV` label "Willkommen"
- Mobile Nav: `MOBILE_NAV` label "Info" für `/willkommen`
- Datei: `src/components/shell/app-shell.tsx`

Empfehlung:

- Gleich benennen (“Willkommen” oder “Info”) – sonst wirkt es wie zwei verschiedene Seiten.

6.3 “Fokusliste” vs “Aufgaben”

Ist-Realität:

- Fokusliste ist **eine gefilterte Sicht** auf offene Tasks (überfällig/heute/auffüllen) auf /heute.
- Aufgaben ist die vollständige Liste /aufgaben.

Verwirr-Potenzial:

- Nutzer könnte denken Fokusliste sei eine separate Liste/Collection.

Empfehlung:

- In Fokusliste-UI einen Subtext wie:
 - “Aus offenen Aufgaben zusammengestellt”
 - und einen klaren Link “Alle Aufgaben (inkl. Erledigt)” (z.B. direkt auf `Status=all`).

Fundstellen:

- Fokusliste Rendering: `src/components/planning/today-dashboard.tsx` (Header “Fokusliste”)
- Aufgaben-Link existiert bereits: Link zu `/aufgaben` in `FocusListCard`

6.4 “Anpassungen” (UI) vs “Adaptive” (intern/legacy)

Fundstellen:

- UI Route: `/anpassungen (src/app/(app)/anpassungen/page.tsx)`
- Legacy redirect: `/adaptive → /anpassungen (src/app/(app)/adaptive/page.tsx)`

Empfehlung:

- Intern/Docs nur noch “Anpassungen” kommunizieren; “Adaptive” nur technisch erwähnen.
-

7) Kurze To-Do Liste (aus diesem Dokument)

- ☐ Fokusliste: Checkbox-Klick mit Confirm oder “Undo/Delay” umsetzen (`today-dashboard.tsx`)
- ☐ Done-Tasks sichtbar machen (TasksScreen default status/Quick Filter verbessern) (`tasks-screen.tsx`)
- ☒ Terminologie vereinheitlicht: „Startansicht“ (`willkommen/page.tsx`, `viewPreferenceRule.ts`, UI Texte)
- ☐ Mobile Nav Label “Info” ↔ “Willkommen” angleichen (`app-shell.tsx`)
- ☐ Fokusliste erklären als “Ausschnitt der Aufgaben” + Link zu “Alle inkl. erledigt”